Computer Performance

Lecture 2

Performance parameters

- Clock frequency
- Average number of instructions per second
- Program execution time
- Number of transactions per second
- Communication channel bandwidth
- Context switch time

Clock frequency

- A measure of performance, it depends on:
  - Integration technology – transistor size and signal path length
  - Supply voltage and the relative distance between logic 1 and 0 levels
- The clock period increases with circuit complexity: RISC computers increase the clock frequency by reducing processor complexity

clock period = delay on the longest path = no_gates * delay_per_gate

Clock frequency

- We can compare computers with the same internal architecture
- For different architectures, clock frequency is less relevant
- Frequency of current processors is 3-5 GHz, saturated due to power dissipation limits

$$P_{dynamic} = \alpha C V^2 f$$

where: α activity factor ∈(0,1), C capacitance, V voltage, f frequency

- Increasing the clock frequency:
  - technological improvement – smaller transistors, through better lithographic methods
  - architectural improvement – simpler CPU, shorter signal paths

Frequency vs processor performance

Processor	Frequency	*SPECint95*	*SPECfp95*
MIPS R10000	195 MHz	11.0	17.0
Alpha 21164	400 MHz	12.3	17.2
UltraSPARC	300 MHz	12.1	15.5
Pentium II	300 MHz	11.6	8.8
PowerPC G3	300 MHz	14.8	11.4
POWER2	135 MHz	6.2	17.6

Average number of instructions per second (IPS)

- Can compare computers with identical or similar instruction sets
- Not suitable for CISC vs. RISC comparison

$$\text{IPS} = \text{f} \times \text{Instructiuni pe ciclu (IPC)}$$

$$\text{IPS} = \frac{1}{\sum_{i=1}^{n} p_i \cdot t_i}$$

p_i=no instr/total no instr

- Cycles per instruction – CPI
  - How many clock cycles it takes to execute an instruction
  - In early computers: 5-120 CPI
  - In recent processors: 1, 0.5, 0.25 CPI – through pipelining and superscalar architecture
- Instructions per cycle – IPC
  - Average number of instructions executed in one clock cycle
- Unit of measure: MIPS (millions of instructions per second), FLOPS, MFlops

uP type	Year	Freq.	MIPS
I4004	1971	0.74MHz	0.09
I80286	1982	12 MHz	2.66
I80486	1992	66MHz	52
P III	2000	600MHz	2054
Intel I7	2011	3.33GHz	177 000
Intel Core I9-9900	2018	4.7GHz	412 000
AMD Ryzen 9	2020	4.3GHz	2 356 000

Program execution time

- Can compare computers with different architectures
- Influenced by the operating system, communication and storage systems
- How do we select a good program for comparison? (a good benchmark)
  - real programs: compilers, encoding / decoding, zip / unzip
  - significant parts of a real program: OS kernel modules, math libraries, graphics processing functions
  - synthetic programs: a mix of instructions in proportions typical for a group of applications (with no real result):
    - Dhrystone – a mix of integer instructions
    - Whetstone - also contains floating-point instructions
- Problems with benchmarks:
  - processor architectures optimized for benchmarks
  - compiler optimization techniques eliminate unnecessary instructions

Other performance parameters

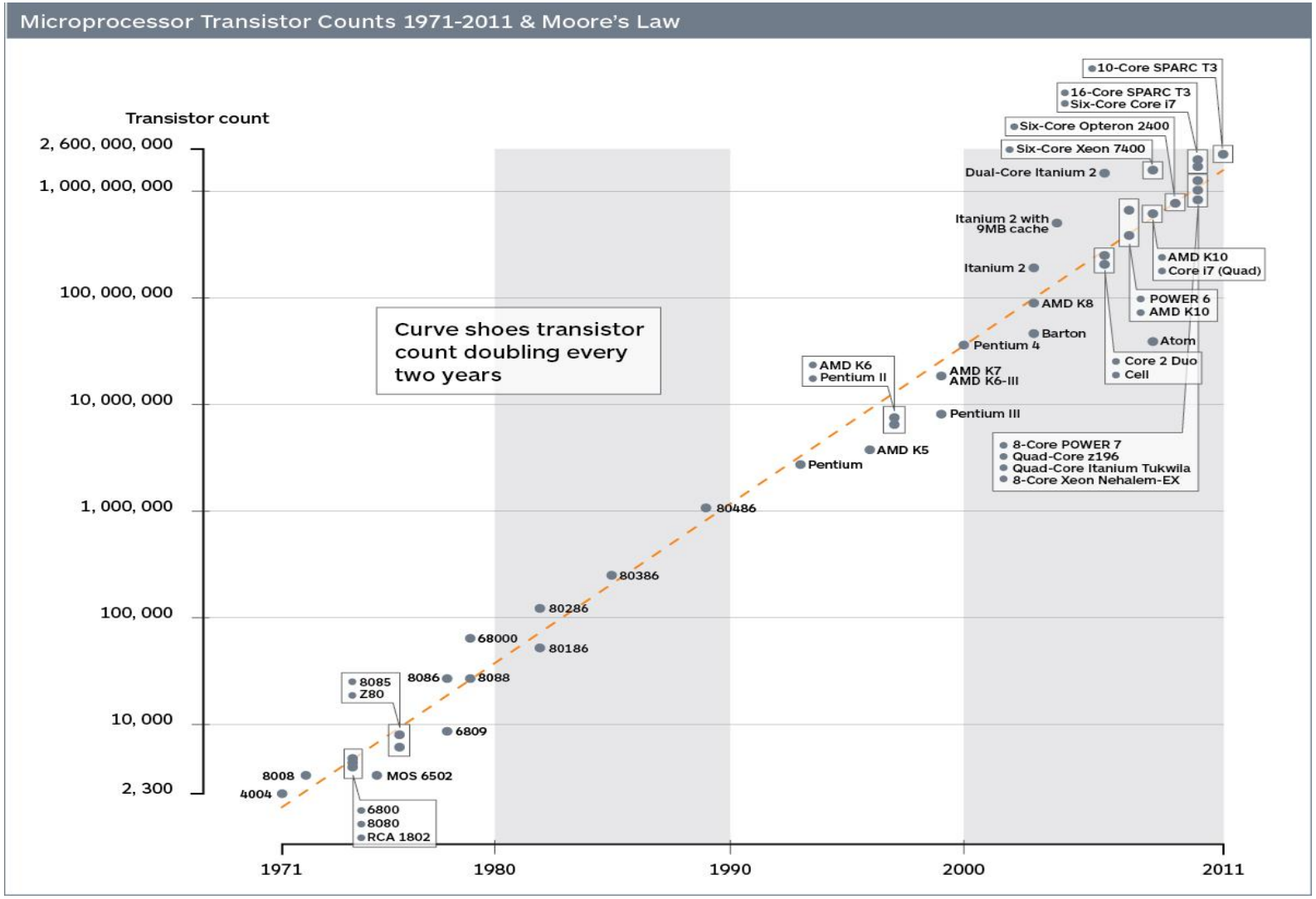
- Number of transactions per second
  - for databases or server systems
  - number of simultaneous accesses to a database or repository
  - operations: read-modify-write, communication, external memory access
  - describes the whole computer system, not only the CPU
- Communication channel bandwidth
  - number of MBytes transmitted per second
  - total bandwidth or useful/usable bandwidth
- Context switch time
  - for embedded and real-time systems
  - example: EEMBC – EDN Embedded Microprocessor Benchmark Consortium

Principles for improving performance

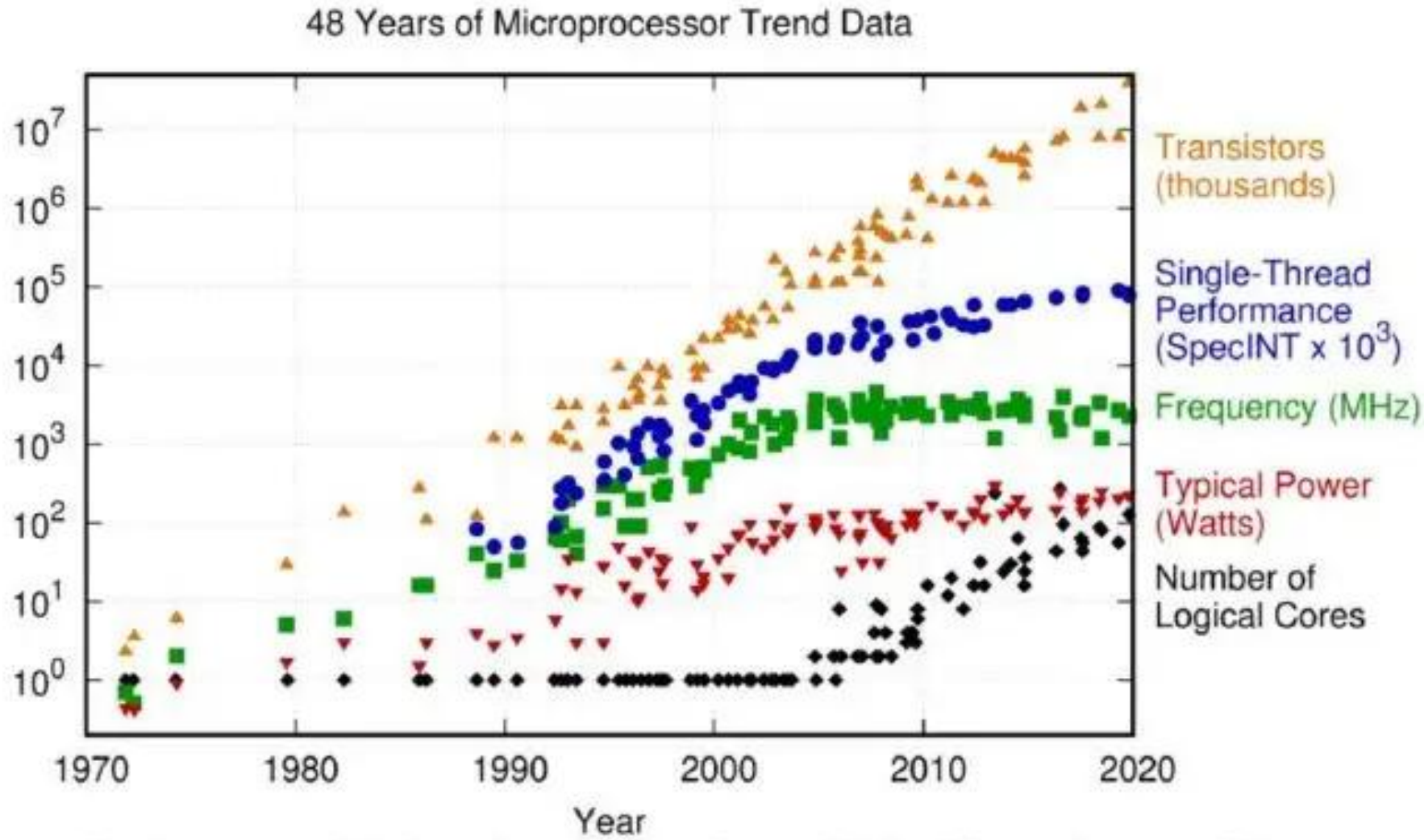
- **Moore's Law**: The number of transistors doubles every 18-24 months
- **Dennard scaling**: with each new technology generation, the area and power consumption of individual transistors are halved, while the power consumption (with twice as many transistors) stays the same.

- Limitations: at small transistor sizes => chip heating, increased power consumption (since 2006 the CPU frequency limit of approx. 3GHz has been reached)

Moore's Law (http://insights.globalspec.com/)



Progress continues through new **architectures** (multi-core, GPUs, TPUs), **3D stacking** and **specialized accelerators**, rather than through simple transistor scaling


48 Years of Microprocessor Trend Data
10^7
10^6
10^5
10^4
10^3
10^2
10^1
10^0
1970
1980
1990
2000
2010
2020
Year
Transistors (thousands)
Single-Thread Performance (SpecINT x 10^3)
Frequency (MHz)
Typical Power (Watts)
Number of Logical Cores
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2019 by K. Rupp

Moore's Law

- Growth will continue, but not at the same pace
- The doubling period is now 6 years; Intel predicts a limit at 2-3 nm
- **Exploration of new materials, technologies, architectures**
- Other similar trends:
  - clock frequency
  - internal memory capacity (DRAMs)
  - external memory capacity (HD, DVD)
  - number of pixels for image and video devices

Semiconductor manufacturing processes
(source: wikipedia**)**

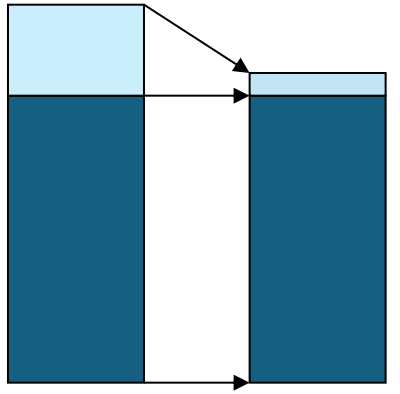
10 µm — 1971
3 µm — 1975
1.5 µm — 1982
1 µm — 1985
800 nm 1989
600 nm 1994
350 nm 1995
250 nm 1998
180 nm 1999
130 nm 2000
90 nm — 2002
65 nm — 2006
45 nm — 2008
32 nm — 2010
22 nm — 2012
14 nm —2014
10 nm — 2016
7 nm — approx. 2018
5 nm — approx. 2020

Principles for improving performance

- Amdahl's Law
- Principle of locality: temporal and spatial
- Principle of parallel execution

Amdahl's Law

- Precursors:
  - the 90/10 principle: 90% of the time the processor executes 10% of the code
  - principle: "make the common case fast"
  - invest more in the parts that matter more
- How do we measure the impact of a new technology?
- Speedup – η – how many times faster the execution is
- **The speedup is limited by the part that cannot be improved**



Old time New time

Numerical example:

f = 0.1; η'=2 => η = 1.052 (5% increase)

f = 0.1; η'=∞ => η = 1.111 (11% increase)

$$\eta = \frac{t_{old_exec}}{t_{new_exec}} = \frac{t_{old_exec}}{[(1-f)t_{old_exec} + \frac{f * t_{old_exec}}{\eta'}}$$

$$\eta = 1 / [(1-f) + f / \eta']$$

Principles of locality

Temporal

- **If a memory location is accessed by the processor, it is highly likely to be accessed again in the near future**
- Explanations:
  - execution of instructions in a loop
  - a variable is used several times in a program sequence
- Consequence: bring the newly accessed memory location closer to the processor for a better access time on a subsequent access => justification for cache memories

Spatial

- **If a memory location is accessed by the processor, neighboring locations are highly likely to be accessed in the near future**
- Explanations:
  - execution of instructions in a loop
  - consecutive access to the elements of a data structure (vector, matrix, record, list, etc.)
- Consequences:
  - bring the neighbors of the location closer to the processor for a better access time on a subsequent access => justification for cache memories and virtual memory
  - transfer blocks of data instead of single locations; block transfer in DRAMs is much faster

Principle of parallel execution

- **When technology limits speed increases, further improvement can be achieved through parallel execution**
- Levels of parallel execution:
  - **data** – multiple ALUs
  - **instructions** – pipeline, super-pipeline and superscalar architectures, Very Long Instruction Word (VLIW)
  - **threads** – multi-cores, multiprocessor systems
  - **application** – distributed systems, Grid and cloud systems
- Parallel execution is one of the explanations for the performance growth of recent processors

Improving processor performance

- Reducing program execution time
- Decrease or increase the values of the parameters in the formulas:

$$t_{exec} = \frac{Nr_instr}{IPS}$$

$$t_{exec} = Nr_instr * CPI * T_{clk} = Nr_instr * CPI / f_{clk}$$

Improving processor performance

- **Increase IPS**

$$IPS = \frac{1}{\sum p_i * t_i}$$ External view

$$IPS = \frac{1}{CPI * T_{clk}} = \frac{f_{clk}}{CPI}$$ Architectural view

- How?
  - reducing instruction duration
  - reducing the frequency (probability) of long, complex instructions (e.g., replace multiplication operations)
  - reducing the clock period/increasing the frequency by simplifying the CPU
- External factors that can influence IPS:
  - code and data access time can drastically influence the execution time of an instruction
  - example: for the same type of instruction (e.g., addition)
    - <1ns for instructions and data in cache memory
    - 15-70 ns for instructions and data in main memory
    - 1-10 ms for instructions and data in virtual memory (HDD)

Improving processor performance

- **Reducing the number of instructions**
- The number of instructions executed by the CPU while running an application
  - improving algorithms,
  - reducing algorithm complexity,
  - more powerful instructions: more operations within a single instruction
- parallel ALUs, SIMD architectures, string operations

Nr_instr = Nr_op / op_per_instr

- Nr_op – the number of elementary operations needed to solve a given problem (application)
- op_per_instr – the number of operations executed in a single instruction (average value)
- increasing op_per_instr may increase CPI (the next parameter in the formula)

Improving processor performance

- **Reducing CPI**
- CPI – cycles per instruction – the number of clock periods needed to execute an instruction
- Instructions have variable CPI; an average value is needed

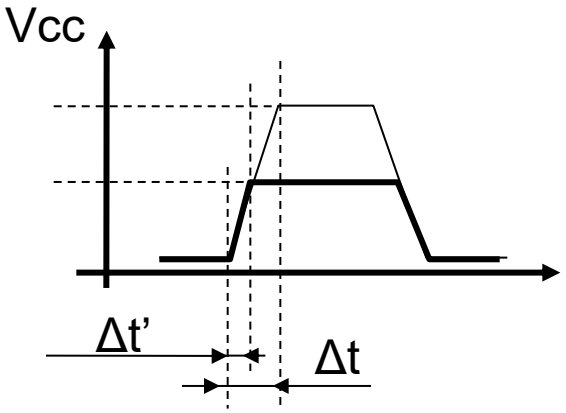
$$CPI_{av} = \frac{\sum n_i * CPI_i}{\sum n_i} = \sum p_i * CPI_i$$

where: ni – the number of instructions of type "i" in the analyzed program sequence
CPI_i – CPI for instruction type “i”

- Methods for reducing CPI:
  - pipelined instruction execution => CPI close to 1
  - superscalar, superpipeline => CPI ϵ (0.25 – 1)
  - Simplifying the CPU and instructions – RISC architecture

Improving processor performance

- Reducing the clock period or increasing the frequency
- Methods:
  - reducing the size of a switching element and increasing the integration density
  - reducing the operating voltage
  - reducing the length of the longest path - simplifying the CPU architecture



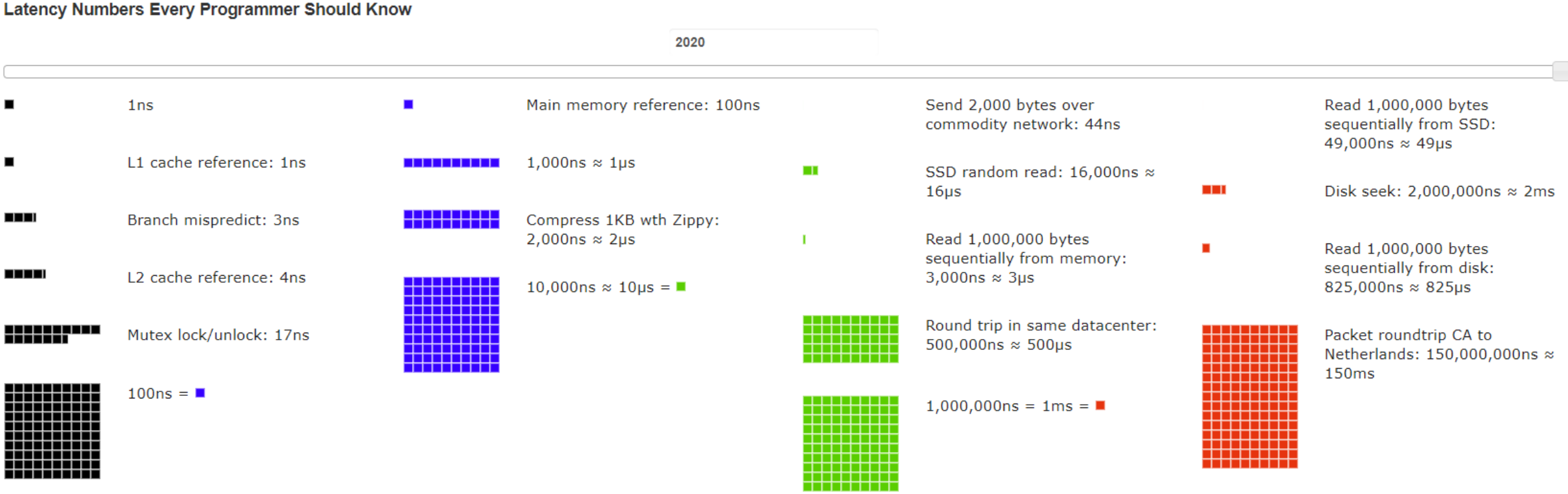
Ways to increase processor speed

- Fewer instructions
- Lower CPI – simpler instructions
- Parallel execution at different levels
- Higher clock frequency

Performance evaluation of a HW+SW system

- How?
  - **Analytical model**: pen-and-paper calculation, profilers
  - **Measurements**: benchmarks
  - **Simulation**: emulators, trace-driven

- Validation – checking model vs measurements / simulation vs measurements



https://colin-scott.github.io/personal_website/research/interactive_latency.html

Original data: Jeff Dean

Example - sorting

- How many / what kind of values
- Algorithm complexity
- What kind of operations
  - Comparisons (branch prediction)
  - Memory access (reads/writes)

- How do you evaluate it?

Measuring computer performance - Benchmarking

- Definition 1 (wikipedia): Benchmarking = the act of running a program, a set of programs or other operations in order to assess the relative performance of a computer, normally by running a number of standard tests.
- Definition 2: Benchmarking = a method for comparing the performance of different computer systems
- Measuring and evaluating the performance of a system is not a trivial task:
  - some computers / processors perform better on some tests and worse on others (e.g., good results for image processing, but less good for database applications)
  - performance must be a weighted average over a number of specific tests

Benchmark programs

- Definition: Benchmark = a test or set of tests used to measure hardware or software performance. These tests provide a reference point for comparing different systems or components.
- Goal: to write a package of programs that best measures the performance of a computer system
- Solutions:
  - real programs - that solve various classic problems
  - synthetic programs - with no practical result, but preserving the instruction frequencies measured in real cases

Characteristics of a good benchmark

- **Representative**: workloads mirror real use cases
- **Clear metrics**: latency, throughput, accuracy, energy
- **Reproducible**: fixed data, run rules, disclosed setup
- **Fair**: vendor-neutral, no benchmark-specific tuning
- ***Scalable****: meaningful from microcontrollers to servers*
- **Portable**: runs across architectures and frameworks
- **Simple**: easy to set up, run and interpret
- **Discriminative**: separates systems beyond measurement noise
- **Statistically rigorous**: repeated runs, variance, warm-up reported
- **Transparent**: open code, data and methodology
- **Hard to manipulate**: hidden test sets, no data contamination
- **Maintained**: versioned, evolving, comparable over time
- **Holistic**: quality and cost together, not speed alone

Benchmark programs

- Micro-benchmarks: measure the performance of a very small, specific piece of code.
- Functional: measure the performance of high-level functions
- Kernel
  - contains code that normally performs a specific basic operation,
  - extracted from the actual program
  - popular kernel: Livermore loops (each loop is a mathematical operation)
  - Linpack benchmark (contains basic linear algebra subroutines); results are reported in MFLOPS
- Synthetic benchmarks
  - Procedure for writing a synthetic benchmark:
    - collect statistics on all types of operations from many application programs
    - obtain the proportion of each operation
    - write a program based on the above proportions
  - Types of synthetic benchmarks:
    - Dhrystone – integer instructions
    - Whetstone – integer and floating-point instructions
- Real programs
  - Word processors (e.g. Word)
  - the user's application software
- Component Benchmarks
  - programs designed to measure the performance of a computer's basic components
  - automatic detection of the computer's hardware parameters such as the number of registers, cache size, memory latency

Benchmark programs

- Other benchmarks
  - I/O benchmarks
  - Database benchmarks: to measure the throughput and response time of database management systems (DBMS)
  - Parallel system benchmarks: used on machines with multiple cores or processors, or on systems consisting of multiple machines
- Issues regarding good benchmarking:
  - some processor architectures were designed for the best benchmark results, but with lower overall performance
  - many benchmarks focus on computation and less on other aspects, such as: memory access time, input / output delays
  - benchmarks are not relevant for large-scale distributed systems
  - there is no single measure of a computer's “performance”

Computing benchmark results

- Arithmetic mean
  - ti – execution time of program "i" in the set of n test programs

$$B_{AM} = \frac{1}{n}\sum_{i=1}^{n} t_i$$

- Weighted arithmetic mean
  - wi – weight of program "i" in the set, indicating its execution frequency
  - wi - chosen so that on a reference computer the execution time of each benchmark (program) is equal => NORMALIZATION

$$B_{AM} = \frac{1}{n}\sum_{i=1}^{n} w_i * t_i$$

- Geometric mean

$$B_{GM} = \sqrt{\prod_{i=1}^{n} t_i}$$

- Normalized geometric mean

$$B_{GM} = \sqrt{\prod_{i=1}^{n} w_i * t_i}$$

Computing benchmark results

- Effects of normalization: the result depends on the machine used as reference: A, B or C

	t on A (s)	*t* on B (s)	*t* on C (s)	*Normalized to A*	*Normalized to B*	*Normalized to C*
Program 1	1	10	100	1 10 100	0.1 1 10	0.01 0.1 1
Program 2	1000	100	10000	1 0.1 10	10 1 100	0.1 0.01 1
Arithmetic mean	500.5	55	550	1 5.05 55	5.05 1 55	0.055 0.055 1
Geometric mean	31.6	31.6	316.22	1 1 31.6	1 1 31.6	0.031 0.031 1

Conclusions from the previous table

Arithmetic mean

- if the reference is computer A:
  - A is as fast as A
  - B is ~5 times slower than A
  - C is 55 times slower than A
- if the reference is computer B:
  - A is ~5 times slower than B
  - B is as fast as B
  - C is 55 times slower than B
- if the reference is computer C
  - A is 18 times faster than C
  - B is 18 times faster than C
  - C is as fast as C

Geometric mean

- if the reference is computer A:
  - A is as fast as A
  - B is as fast as A
  - C is ~32 times slower than A
- if the reference is computer B:
  - A is as fast as B
  - B is as fast as B
  - C is ~32 times slower than A
- if the reference is computer C
  - A is ~32 times faster than C
  - B is ~32 times faster than C
  - C is as fast as C

Computing benchmark results

- Advantages of the geometric mean:
  - It is independent of the running times of the individual programs
  - It does not matter which machine is used for normalization
- Disadvantage of the geometric mean: it does not predict execution time

Examples

- The Whetstone synthetic program
  - Published in 1976 by the National Physical Laboratory (NPL), UK
  - preserves the instruction frequencies of scientific and engineering applications
  - written in Algol and later in Fortran and Pascal
  - floating-point instructions play an important role
- The Dhrystone synthetic program
  - Published in 1984
  - preserves the instruction frequencies of system programming (e.g., operating system components) using the Ada and C programming languages.
  - frequency measurements are published with no emphasis on floating-point operations
- Problems with synthetic benchmarks:
  - they do not reflect the needs of a real application well
  - some computer architectures were optimized for the best performance on synthetic benchmarks, but with lower performance on real applications

Examples

- Kernel benchmark
  - based on the time-critical components of real applications
  - focused on measuring the performance of supercomputers running scientific applications
  - Livermore Loops:
  - benchmark for parallel computers
  - 24 “do” loops performing various mathematical operations (e.g. solving a set of linear equations, matrix operations in hydrodynamics, etc.)
  - Linpack: performs numerical linear algebra
- SPEC - Standard Performance Evaluation Corporation www.spec.org
  - international non-profit organization focused on developing standard tools for measuring computer system performance
  - develops standard benchmark suites based on real applications
  - they include source code
  - they have tools for generating performance reports

Examples of benchmarks

- **Processors**: SPEC CPU 2017, SPECpower_ssj2008 (performance and energy efficiency)
- **HPC**: HPL (TOP500), HPCG, Graph500, Green500
- **Databases**: TPC-C (transactions), TPC-H and TPC-DS (analytics), YCSB (NoSQL)
- **Cloud and microservices**: DeathStarBench, CloudSuite
- **Embedded systems**: EEMBC CoreMark, ULPMark (ultra-low power), MiBench
- **ML training**: MLPerf Training (MLCommons)
- **ML inference**: MLPerf Inference (datacenter and edge), MLPerf Mobile
- **Edge AI / TinyML**: MLPerf Tiny (microcontrollers: latency and energy)
- **Language models**: MMLU, GPQA, SWE-bench, HumanEval, HELM
- **Computer vision**: ImageNet, COCO